Investigating the use of Reinforcement Learning Algorithms in Traffic Environments

Abdul Muhaymin Abdul Hafiz[1]

BSc Computer Science

Supervisor: Prof. Graham Roberts

Submission date: Day Month Year

[1]**Disclaimer:** This report is submitted as part requirement for the BSc Degree in Computer Science at UCL. It is substantially the result of my own work except where explicitly indicated in the text. The report may be freely copied and distributed provided the source is explicitly acknowledged.

Abstract

Summarise your report concisely.

Contents

1 Introduction **2**

1.1 The Problem . . . 2

1.2 Aim and Objectives . . . 2

1.3 Report Overview . . . 3

2 Literature Review **4**

2.1 Fixed-Time Control . . . 4

2.2 Vehicle-Actuated Control . . . 4

2.3 Adaptive Traffic Control . . . 4

2.4 Limitations . . . 5

2.5 Motivation for Reinforcement Learning . . . 5

2.6 Summary . . . 6

3 Simulated Environment **7**

3.1 OpenStreetMap . . . 7

3.2 Simulation of Urban MObility . . . 8

3.3 Route Generation . . . 9

3.3.1 Binomial Distribution . . . 10

3.3.2 Poisson Distribution . . . 10

3.4 Summary . . . 11

4 Reward System **12**

4.1 Baseline Environment . . . 12

4.1.1 Traffic Light System . . . 12

4.1.2 Detectors . . . 13

4.1.3 Traffic Parameters . . . 14

4.2 Reward Calculation . . . 15

4.2.1 Traffic Control Interface . . . 15

4.2.2 Reward Formula . . . 15

4.2.3 Weights . . . 16

4.3 Summary . . . 17

5 Q-Learning **18**

5.1 Markov Decision Process . . . 18

5.2 Q-Learning Algorithm . . . 18

5.3 Performance . . . 18

6 Deep Q-Learning **19**

7 Communicative Deep Q-Learning **20**

8 Supplementary Analysis **21**

9 Conclusions **22**

9.1 Achievements 22

9.2 Evaluation 22

9.3 Future Work 22

A Other appendices, e.g., code listing **25**

Chapter 1

Introduction

1.1 The Problem

Traffic lights are ubiquitous signalling devices used to control the flow of vehicle and pedestrian traffic, primarily for safety and better regulation of traffic. First introduced in 1868 on Parliament Square in London [Tha17], there have been a wide range of designs created with the most common being a three-colour design – that is green, amber, and red.

The three-colour de jure standard typically uses an internal clock to cycle through and control the duration of each signal phase in a pre-programmed pattern. Whilst this is an obvious approach, the fixed timer takes for granted that traffic demand will remain relatively constant over time, disregarding finer granularity and variations in conditions.

In an endeavour to improve upon this, advanced traffic light systems equipped with sensors are progressively being introduced. These systems dynamically adapt signal timings in response to observed traffic conditions by using rule-based logic and manually designed heuristics. This approach offers improvements in waiting times and vehicle throughput [ASC17], however it relies on predefined rules defined by domain experts that may fail to generalise and perform optimally in all conditions.

1.2 Aim and Objectives

The overall aim of this dissertation will be to perform a detailed analysis and evaluation of how an intelligent agent may be used to hopefully improve upon existing traffic light systems. Rather than explicitly defining behaviour in different traffic scenarios as before, we will observe how optimal decision-making policies may be learnt independently through the use of reinforcement learning algorithms.

Based on this aim, the following objectives are defined:

1. Define an environment in which our simulations and experiments will be conducted.
2. Build an abstracted computer representation of the area.
3. Review the traffic light systems currently in use.
4. Develop our own intelligent agents progressively, going through various reinforcement learning algorithms to see what improvements are made.

5. Consider the impact on both pedestrians and vehicles, and the effects of changing the road design in itself as a means of making improvements.

6. Evaluate the performance of our intelligent agents.

7. Finally, assess the feasibility of our agents and what factors would need to be considered in a tangible implementation.

Time is a currency that once spent cannot be regained. Traffic lights govern our lives to the extent that saving mere fractions of a second at the individual level can compound to an indescribable amount saved when considering the aggregate. By leveraging intelligent agents, this research hopes to take traffic lights a step further – regardless of how big that step may be.

1.3 Report Overview

!!! MUST REDO SINCE THE STRUCTURE HAS CHANGED !!!

We will begin with defining the environment in which we will be conducting our traffic experiments, and then building out this environment using available open source tools and libraries. Whilst accuracy and scope of the environment are important, they will not take precedence over the implementation and analysis of the intelligent agents themselves.

Following this, the context and theory behind reinforcement learning itself will be explored, as well as definitions and terminology that will be used throughout the dissertation. These foundations will be necessary when explaining the motivation behind decisions made later on.

Next we will discuss the creation of an application programming interface that will enable us to connect to our environment for the purposes of observation and control. Based on the default environment and traffic light systems, we will also define a reward which will be used to evaluate performance.

Upon making our environment, we will implement and explore different reinforcement learning algorithms our intelligent agent may use – namely Q-learning and Deep Q-learning. These discussions will be supplemented with the theory regarding how each works, as well as the improvements and limitations proposed by each. Further derivatives of these algorithms and the advantages they bring will also be briefly touched upon, however they will not be implemented.

Continuing from this, a novel approach in which multiple intelligent agents interface together through the use of a network layer will be detailed and implemented. The limitations imposed by such an approach will be considered, as well as how it may be implemented in practice outside of the simulated environment setting.

Finally, after testing and evaluating the different approaches in unforeseen traffic scenarios, we will draw our overall conclusions. Feasibility of such a project will be discussed, as well as interesting technical challenges that were encountered. Future work and limitations to consider will then be commented on.

Chapter 2

Literature Review

Modern traffic light control has changed significantly over the years, ranging from simple fixed-time mechanisms to complex adaptive and coordinated systems. Broadly speaking, systems can be categorised into three main types: fixed-time control, vehicle-actuated control, or adaptive traffic control. These will explored in this chapter, as well as the motivations for introducing intelligent agents.

2.1 Fixed-Time Control

Fixed-time traffic signals operate according to a predetermined schedule. Signal phases and cycle lengths are configured in advanced, typically using historical data [GBG]. The controller cycles repeatedly through these phases regardless of traffic conditions.

Such systems are predictable and inexpensive, used in city centers, interchanges, or roadworks. However, they assume stable traffic patterns. During periods of unusual demand, fixed-time systems cannot respond dynamically, leading to inefficient utilisation of junctions.

2.2 Vehicle-Actuated Control

Actuated systems are an improvement over purely fixed-time control systems. These systems utilise sensors – such as inductive loop detectors or cameras – to detect vehicles. Pedestrians can also use push buttons to register their presence [DfT23]. This may for example enable one route at an intersection to stay active if other routes are barren.

Typically used in rural roads or highway operations, they may be semi-actuated, where detectors are only placed on side approaches, or fully actuated, in which detectors are installed on all approaches. Overall, responsiveness to traffic conditions is improved, reducing unnecessary waiting. However, it relies on rule-based logic and manually defined heuristics, which may fail to perform optimally in all conditions [Li23].

2.3 Adaptive Traffic Control

Unlike fixed-time or purely vehicle-actuated control systems, adaptive traffic control enables signal timings to be adjusted in real time based on observed traffic conditions. Data is collected from inductive loops, cameras, or other technologies.

By modifying parameters such as cycle length, phase splits, and offsets in response to traffic [Li23], adaptive controllers seek to reduce delays, smooth traffic flow, and increase throughput (though it may also be used to *discourage* high volumes of traffic at the request of local residents for example).

Split Cycle Offset Optimisation Technique (SCOOT) and the Sydney Coordinated Adaptive Traffic System (SCATS) are two examples widely deployed in urban areas. These systems group signals into regions and make dynamic adjustments to balance delays and improve coordination. This helps promote "green waves", where vehicles encounter consecutive green lights without having to stop, but possibly with added pedestrian and cyclist delays [CU21].

SCOOT has demonstrated an improvement in traffic performance of order of 15% compared to fixed-time control systems [ITS12]. In London, with the introduction of the cloud-based Real Time Optimiser platform, this is being replaced with FUSION – intended to bring even further improvements. However, these systems still rely on human-designed algorithms. Optimal strategies are not necessarily "learnt", nor does it adapt to every possible traffic scenario without explicit modeling by engineers.

2.4 Limitations

Existing traffic light systems work well enough, however there are several fundamental limitations.

Firstly, these systems rely on parameter tuning and predefined logic. Even for adaptive traffic control the optimisation strategies used are engineered by domain experts. Performance is constrained by assumptions embedded in the design and, when traffic conditions deviate from these, optimality is not guaranteed.

Secondly, most traditional systems are designed such that they prioritise vehicular throughput. Pedestrians and cyclists are typically the secondary optimisation target. This may lead to scenarios in which vehicles reap the benefits at the expense of pedestrians who may see increased waiting times.

Thirdly, these systems are limited with regards to learning. Although adaptive traffic control systems respond in real time, they do not learn from historical experience in the sense of improving their strategy. Parameters may be recalibrated periodically by engineers, but this is not autonomous and novel strategies may be missed. Considering the scale of these systems, it may be hard to consider all possible variables.

Finally, implementation and maintenance costs are non-trivial. These systems require extensive sensor and communication infrastructure, calibration, as well as continuous monitoring by traffic engineers. The complexity of these systems also make them less transparent and harder to validate formally with respect to performance benefits.

2.5 Motivation for Reinforcement Learning

!!! ADD SHORT SECTION ABOUT EXISTING REINFORCEMENT LEARNING + TRAFFIC LIGHT !!!

Reinforcement learning offers an alternative method of traffic signal control. Control policies are not explicitly programmed, but instead emerge through intelligent agents interacting with a traffic environment. An intelligent agent can observe current traffic state, select a signal phase action, and receive feedback in the form of a reward from the environment.

Over time, behaviour gradually improves and a control strategy emerges through experience, as opposed to encoding behaviour manually for every possible scenario. The ability to learn makes this very powerful; most limitations discovered above are mitigated by learning – though perhaps not the infrastructure costs. Ergo, this will be explored for the remainder of this dissertation.

2.6 Summary

This chapter described the fixed-time control, vehicle-actuated control, and adaptive traffic control systems in detail, mentioning the benefits and drawbacks of each. The fundamental limitations of these systems as a whole were also explored, motivating why a novel approach which uses reinforcement learning is desirable.

Chapter 3

Simulated Environment

Before we get to our intelligent agents, we must first construct an environment where the agent can receive input – or state, rather – and perform actions. For the sake of simplicity this environment will be constrained to the streets surrounding University College London's (UCL) Bloomsbury Campus. This area provides scope sufficient for the agents to train well and generalise to other situations.

3.1 OpenStreetMap

OpenStreetMap is a map database maintained by a community of volunteers. Using their data obtained from ariel photo imagery, satellite imagery, geodata sources, as well as manual community additions [HW08], we can extract a portion of UCL's Bloomsbury Campus[1], also complete with data such as traffic lights and crossings.

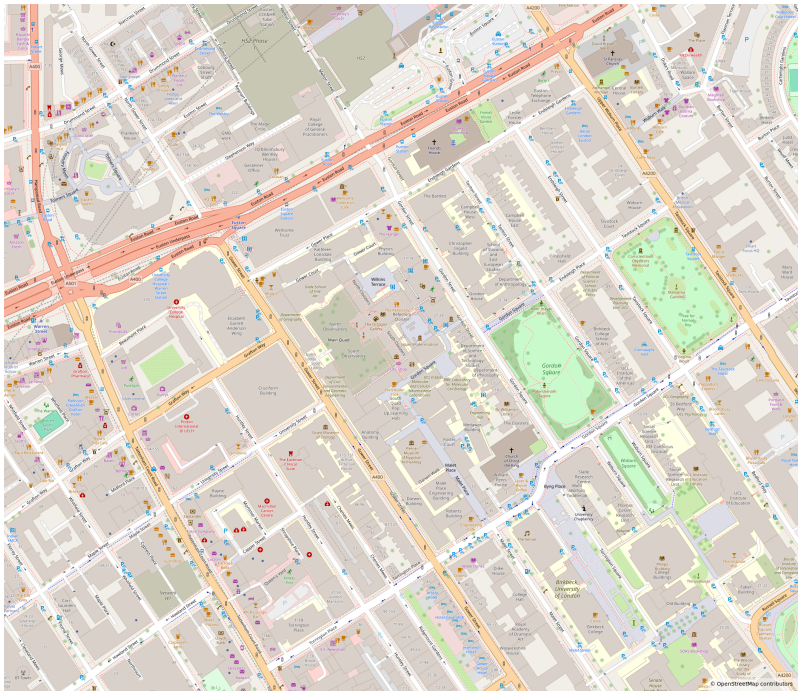


Figure 3.1: Map of UCL's Bloomsbury Campus extracted from OpenStreetMap data.

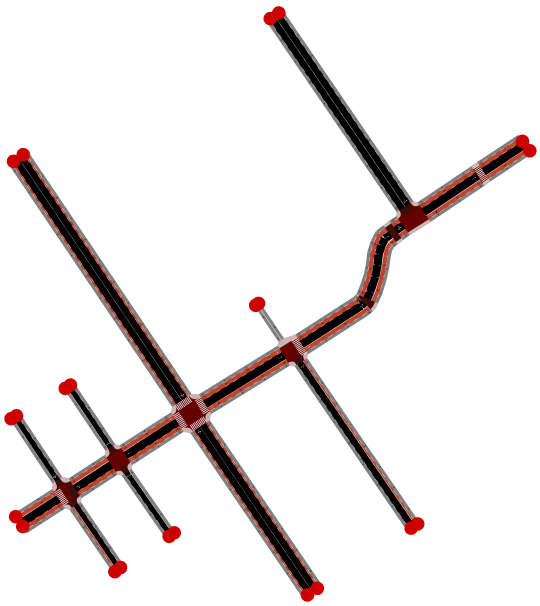
[1]The map centre is: 51.52519°N, 0.12870°W (WGS84).

Although the data from OpenStreetMap provides a lot of useful information, we will supplement it with Google Street View – featured in the Google Maps software – which provides interactive panoramas from positions along the streets. This will allows us to find out more precise information such as lane markings and widths.

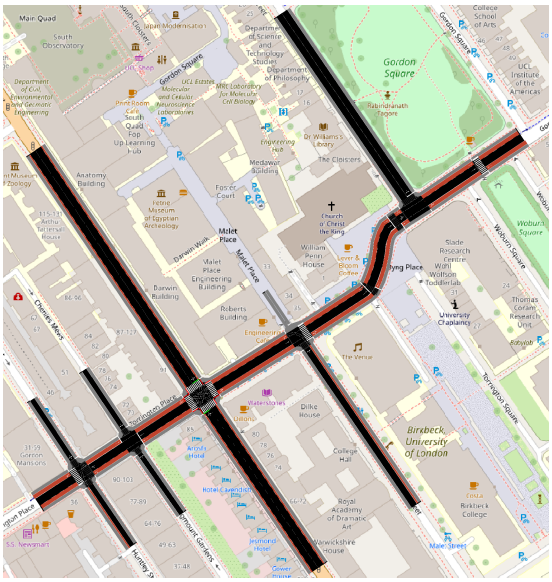
3.2 Simulation of Urban MObility

Simulation of Urban MObility (SUMO) is an open source, multi-modal traffic simulation package designed to handle large networks. It allows for intermodal simulation including pedestrians and comes with a large set of tools for scenario creation.

Using the image in Figure 3.1, we can create a network using the *netedit* graphical network editor included in SUMO. Networks are primarily comprised of edges, which is subdivided into lanes, and junctions, which are nodes where the edges meet. Junctions additionally support crossings, lane connections, and traffic lights.



(a) Road network in netedit.



(b) Simulation view in SUMO.

Figure 3.2: Demo simulation environment.

In Figure 3.2a, the red circles and intermediary maroon links indicate junctions, whereas the in between sections are the edges. Each edge can be comprised of a bicycle lane, pedestrian lane, and/or vehicle lane as shown. Additionally, crossings have been added to the junctions, and a configurable traffic light system has been added with the lane connections appropriately adjusted.

Though not visible in the road geometry, lane and junction priorities have also been set, consistent with rules 170–183 of The Highway Code [HWC]. In the UK, 'main' and 'side' roads are relative terms, with the road with priority (e.g. no give-way markings, or priority road signs) functioning as the main road, while the road required to give way is the side road.

!!! MENTION ERRORS IN CREATING SIMULATION AND HOW IT WAS FIXED – BICYCLE LANES OVERLAPPING !!!

3.3 Route Generation

The SUMO installation includes an additional collection of tools for convenience. One such tool is a `randomTrips.py` script written in Python which enables automatic generation of trips and routes[2] for different origin-destination pairs for a given road network.

We use this script to generate traffic for the following three types of entities, though SUMO provides the flexibility for more:

- Cars
- Bicycles
- Pedestrians

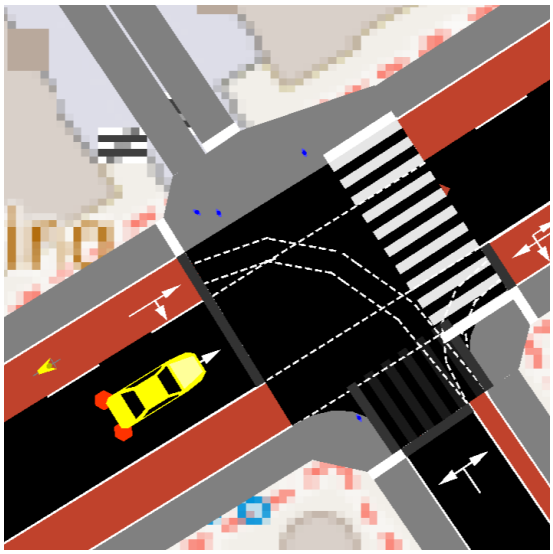


Figure 3.3: Example of car, bicycle, and pedestrian entities.

In Figure 3.3, the car, travelling east, is shown as a yellow rectangle with brake lights; the bicycle, travelling east on the bicycle lane, is depicted as a yellow trapezium-like shape emerging from the left side of the image; and the pedestrians are represented as blue circles.

Route generation is controlled by several parameters, including:

- `--seed`: Ensures reproducibility of the random generation process.
- `--insertion-density`: Controls the density of generated entities.
- `--binomial`: Controls how vehicle insertions are distributed over time by sampling departures from a binomial distribution.
- `--fringe-factor`: Ensures that vehicles spawn at the outer edges of the network.
- `--random-factor`: Randomizes edge weights to increase the variability between generated routes.

There are other obvious parameters omitted which allow you to specify information such as the input network for which the routes are generated, the output trip and route files, the entity type, and the start and end time of entity generation.

[2]N.B. a *trip* defines a random start A and end B, whereas a *route* defines the exact edges to traverse to get from that start A to end B.

3.3.1 Binomial Distribution

The `--binomial` parameter allows vehicle departures to be sampled from a Binomial distribution. Real-world traffic arrivals are not perfectly periodic due to human behaviour as well as variations in the environment. Rather than inserting vehicles into the simulation at perfectly regular intervals, stochastic variation is introduced such that some seconds contain more vehicle departures and others fewer.

$$X \sim \text{Binomial}(N, p)$$

$$\mathbb{E}[X] = Np$$

Where N is the number of trials given by the `--binomial` parameter, p is the probability of success, and Np equals the insertion rate calculated from the `--insertion-density` parameter. Thus, the overall expected value of departures per time step still remains equal to the deterministic insertion rate.

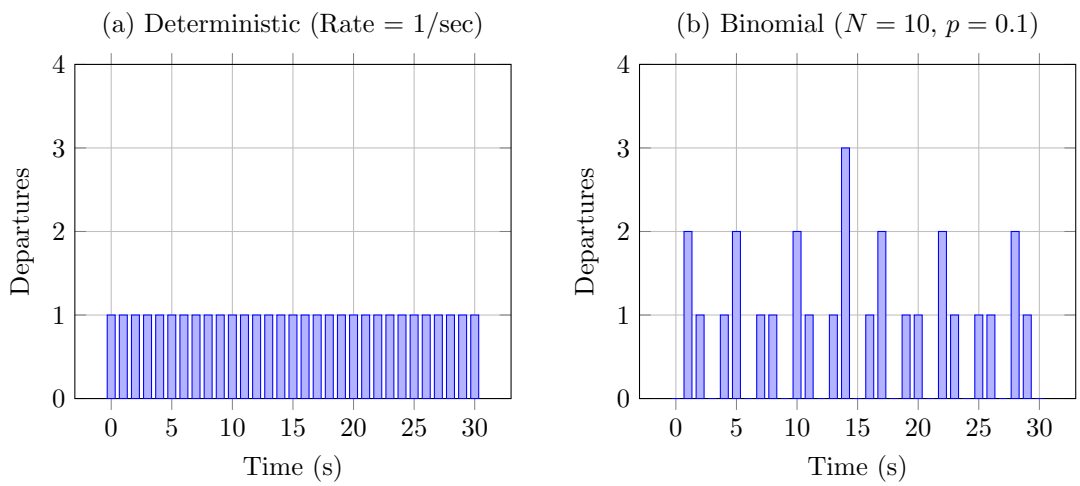


Figure 3.4: Comparison of deterministic and binomial vehicle insertions.

3.3.2 Poisson Distribution

In traffic theory, Poisson processes are commonly used and preferred over the Binomial distribution to model vehicle arrivals [GS55]. In fact, for sufficiently large N and small p, the binomial distribution approximates a Poisson distribution:

$$\text{Binomial}(N, p) \approx \text{Poisson}(\lambda), \quad \text{where} \quad \lambda = Np$$

The `randomTrips.py` script provides flexibility in defining vehicle traffic. For example, you may define there to be n fixed 'flows' which traffic may take – some of which may annoyingly be duplicates – or you let any of the possible routes for the traffic be arbitrarily, independently chosen each time a new vehicle is inserted.

Whilst there is no problem with the Poisson distribution per se, the script is designed such that it is only compatible with the prior definition method. Binomial sampling on the other hand is compatible with the latter method as well as other parameters which are more flexible for our

purposes and have less implementation-specific issues. For this reason, it was chosen.

3.4 Summary

This chapter described the creation of a simulated environment based on the streets surrounding UCL's Bloomsbury campus. Road network data was extracted from OpenStreetMap and refined using Google Street View. The network was implemented using SUMO, where edges and junctions were defined.

Using SUMO's `randomTrips.py` tool, there is a way for scalable creation of diverse traffic scenarios. Stochastic vehicle insertions is used to better approximate real-world traffic variability. Overall, these components establish a way in which we can train and evaluate intelligent agents.

Chapter 4

Reward System

Now that we have seen how an environment – composed of a network and traffic routes – is constructed, we will develop a numerical reward system used to evaluate the performance of different intelligent agents. This reward system will use a contemporary traffic light system algorithm as a baseline; a good reward system is paramount to training intelligent agents effectively.

4.1 Baseline Environment

It is crucial that we understand the performance of existing traffic light systems so we can make an accurate judgement as to the benefits of our intelligent agents. Using the demo network created previously in Figure 3.2, consisting of one traffic light system (TLS), three zebra crossings, and several regular crossings, we can create a baseline environment.

4.1.1 Traffic Light System

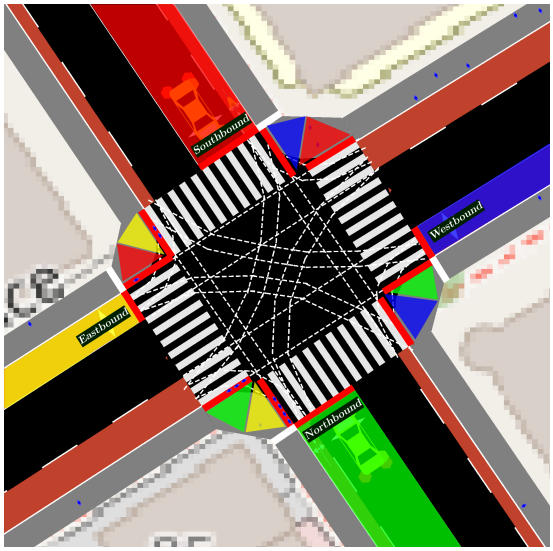


Figure 4.1: Traffic Light System in the demo network.

In Figure 4.1 above, green represents traffic – including pedestrians – that is northbound; blue represents westbound traffic; red represents southbound traffic; and yellow represents eastbound traffic.

In London, adaptive traffic light control systems are widely used enabling coordination between traffic lights – as mentioned in our literature review. SUMO unfortunately does not support this system natively, and building a controller replica goes beyond the scope of this dissertation. Furthermore, there is only one TLS in this demo network so there wouldn't be any multi-TLS coordination regardless.

For the sake of simplicity, the TLS will use fixed-time control. The general practice is for the cycle length to be between 50 and 120 seconds, with 90 seconds often being considered optimum [SIIG04]. Below is a simple cycle striking a balance between safety and operational performance.

Phase	**Duration (s)**	**Approach**	**State**
1	30	Northbound/Southbound	Green
2	3	Northbound/Southbound	Amber
3	2	All approaches	Red
4	30	Eastbound/Westbound	Green
5	3	Eastbound/Westbound	Amber
6	2	All approaches	Red
7	10	Pedestrians	Green
8	10	All approaches	Red
Total	**90**		

Table 4.1: Fixed-time control phase structure.

4.1.2 Detectors

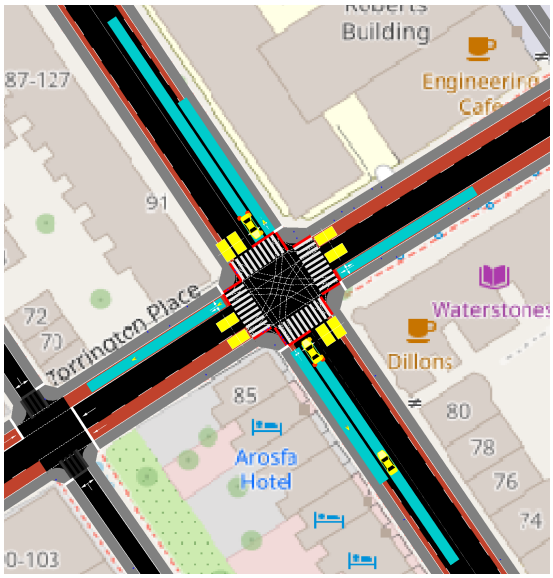


Figure 4.2: Detectors in the demo network.

In Figure 4.2 above, the yellow-coloured detectors placed at the departure lanes are induction loops, and the aqua-coloured detectors placed at the approaches are lane area detectors.

A step length – the time between each step – in our simulation is defined as 1 second. After each simulation step, the detectors can be queried for information such as the vehicle count passing through them, mean speed, unique identifiers of the vehicles themselves, and time since last detection.

Induction loops have a fixed length, making them ideal for measuring vehicle throughput. Lane

area detectors – perhaps implemented as a vehicle tracking camera in reality – can cover a larger area, making it tailored for measuring queues of traffic and keeping track of wait times.

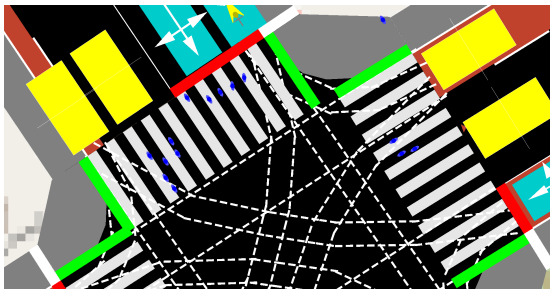


Figure 4.3: Example of pedestrians crossing the road.

SUMO has no explicit support for pedestrian detectors, however a pseudo-detector can be made. To measure the queue of pedestrians waiting to use a crossing, pedestrians who are on the walking edges adjacent to the crossing are filtered such that only the pedestrians whose next walking edge is the crossing itself remain.

Pedestrians about to exit a crossing is measured by retrieving all pedestrians currently using that crossing and checking whether the remaining length of road before they finish crossing is below a threshold.

4.1.3 Traffic Parameters

In our baseline environment, the traffic densities for cars, bicycles, and pedestrians were chosen based on anecdotal observations[1] that they were moderate, emulating an average day-to-day scenario – not too light, not too dense. Furthermore, they were generated with the `--binomial` parameter set with $N = 100$ to approximate a Poisson distribution as explained previously.

The `--insertion-density` parameter, which specifies the number of entities per hour per kilometer of road, was assigned for each entity type such that the traffic densities for road users, i.e. the cars and bicycles which use road lanes, had a combined density equivalent to pedestrians.

Entity	**Type**	**Insertion Density (per hr/km)**	**Relative to Pedestrians**
Car	Road User	100	0.5×
Bicycle	Road User	100	0.5×
Pedestrian	Pavement User	200	1.0×

Table 4.2: Comparison of the insertion density of the entities.

It is important to note that in reality, in a dense urban environment such as London, there tends to be many more pedestrians to other road users, and there is typically more cars than bicycles. But, for the purposes of this dissertation, these nuances are ignored.

The parameters used in route generation for our baseline environment can be summarised and simplified as follows:

- `--insertion-density`: 100 for cars and bicycles respectively, 200 for pedestrians ($100 + 100 \equiv 200$).
- `--binomial`: $N = 100$, where N is the number of trials.

[1] Not the best approach, but "*I know it when I see it*" [PS64].

- `--fringe-factor`: Max – vehicles are only inserted from the very edges, rather than middle of the road. Disabled for pedestrians.
- `--random-factor`: None – all active edges are given equal weighting when determining which to insert a new vehicle.

4.2 Reward Calculation

Using the `randomTrips.py` script with the baseline environment and parameters defined above, we can generate traffic routes. We generate 1000 distinct routes – each comprising of cars, bicycles, and pedestrians – in total for the demo network, each one having a generation period of 1 hour (3600 seconds). A seed is also set for each, enumerating from 1, to ensure variability and reproducibility in the routes.

4.2.1 Traffic Control Interface

The SUMO package provides an interface that enables us to connect to our environment simulation – called "*sumo*" itself – for the purposes of observation and control. Known as Traffic Control Interface (TraCI), it uses a TCP based client/server architecture, where *sumo* is the server connected to by clients.

TraCI also provides a Python module of the same name, where you can create clients which connect to the environment simulation that is spun up. This is what the rest of the algorithms and systems in this dissertation will be built upon.

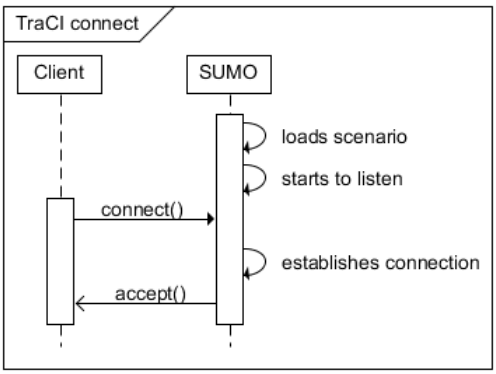


Figure 4.4: TraCI: establishing a connection to SUMO [Tr26].

Using TraCI, we run our baseline environment with the 1000 distinct routes generated previously in turn. For each route, the simulation is executed for 3600 seconds producing a total of $1\,000 \times 3\,600 = 3\,600\,000$ simulation steps across all runs.

4.2.2 Reward Formula

The reward formula combines four measurable indicators of traffic performance into a single scalar value that can be used to evaluate performance, and can be optimised by an intelligent agent. They are as follows, calculated using the aforementioned detectors queried using TraCI:

- z_{vd}: Total vehicle delay

- z_{vt}: Total vehicle throughput
- z_{pd}: Total pedestrian delay
- z_{pt}: Total pedestrian throughput

Delay is defined as the continuous time (seconds) in which vehicles or pedestrians in the detection area have had a speed of less than 0.1 m/s, indicating waiting. Vehicle throughput is defined as the number of vehicles crossing the departure detectors; pedestrian throughput is defined as the number of pedestrians finishing using their crossing.

At time step t, the reward R_t can be expressed as a weighted sum of the four metrics:

$$R_t = w_{vd}(-z_{vd}) + w_{vt}(z_{vt}) + w_{pd}(-z_{pd}) + w_{pt}(z_{pt}) \tag{4.1}$$

Where the coefficients, or weights rather, w_{vd}, w_{vt}, w_{pd}, and w_{pt} determine the relative importance of each metric. Delay terms are negated since increased delay represents worse performance.

4.2.3 Weights

The four metric values z_{vd}, z_{vt}, z_{pd}, and z_{pt} are calculated per each of the 3 600 000 simulation steps and then cached inside of a buffer with four columns. Once the final route has finished running, we can use the `numpy` Python module to calculate statistics regarding each metric stored within the buffer.

Metric	Mean	Standard Deviation	Min	Max	Sum
z_{vd}	63.07	67.07	0	5032	227066504
z_{vt}	0.235	0.515	0	5	844528
z_{pd}	211.69	220.43	0	1702	762082178
z_{pt}	0.252	1.199	0	21	908941

Table 4.3: Summary statistics for vehicle and pedestrian metrics (runtime: 3:16:02).

These metrics are not for a single vehicle or pedestrian, but rather it is based on the cumulative value for all vehicles or pedestrians that are on the detectors at a given simulation step.

Since the fixed-time control TLS will serve as the baseline against which we evaluate our intelligent agents, the weights in formula (4.1) are adjusted based on the summary statistics in Table 4.3 above. Namely, each of the weights are normalised using the mean value for the corresponding metric such that the average calculated reward for the fixed-time control TLS is a baseline of 0.

- w_{vd}: $\frac{1}{63.07} = 0.01586$
- w_{vt}: $\frac{1}{0.235} = 4.255$
- w_{pd}: $\frac{1}{211.69} = 0.004724$
- w_{pt}: $\frac{1}{0.252} = 3.968$

If we calculate the reward after each of the 1000 routes is run using the fixed-control TLS, we confirm that the average calculated reward is about 0.

!!! RERUN EXPERIMENT TO GET CORRECT GRAPH !!!

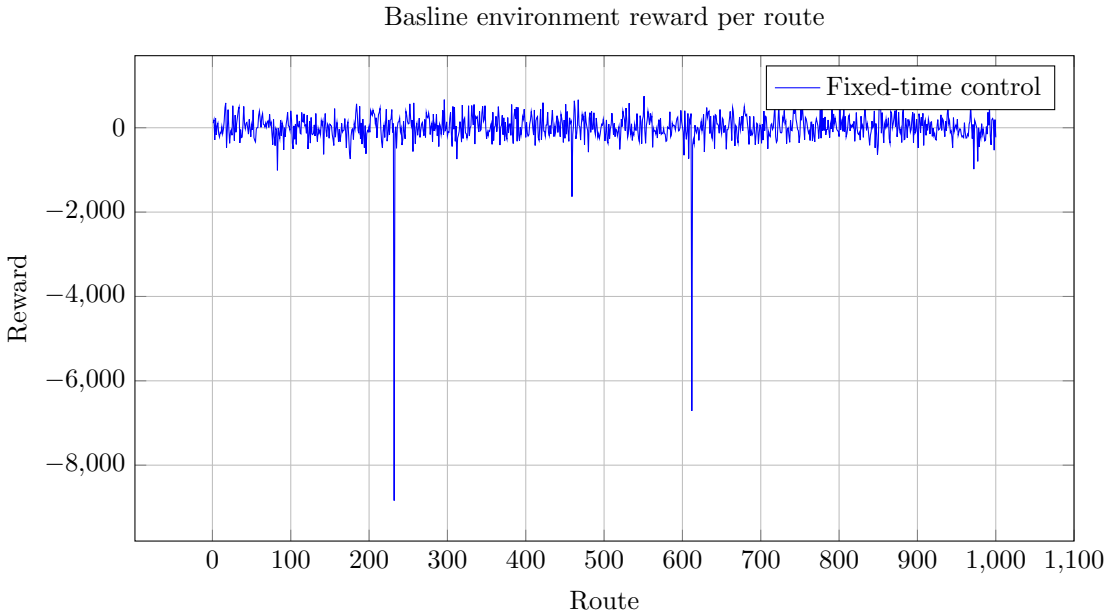


Figure 4.5: Reward per route using a fixed-time control TLS.

Equal weighting is currently given to pedestrians and road users (cars and bicycles) as a consequence of the traffic parameters mentioned previously for our baseline environment and the current reward formula. Neither is implicitly being prioritised over the other.

Although not explored further in this dissertation, an interesting extension would be to vary the weights to prioritise either pedestrians or vehicles. For example, vehicles may reasonably be assigned a slightly higher priority considering that cars in England have an average occupancy of approximately 1.57 persons per [Sta19]. However, this may conflict with broader urban plans of encouraging walking and improving pedestrian safety.

4.3 Summary

This chapter explored the creation of a baseline environment that incorporates a fixed-control traffic light system as well as well defined traffic parameters. To observe car, bicycle, and pedestrian traffic, a set of detectors was deployed.

Using the TraCI interface, a way for controlling and observing the simulation environment was created. This also enabled a weighted reward formula to be defined and calculated. Both of which will be crucial for our following intelligent agents.

Chapter 5

Q-Learning

Having created a baseline environment with a traffic light system which can be observed and controlled, as well as creating a reward system, we now create the first of our intelligent agents. The context of reinforcement learning and the Markov decision process will first be explored before creating an implementation of the Q-learning algorithm. The performance will then be evaluated, and the next steps will be considered.

5.1 Markov Decision Process

5.2 Q-Learning Algorithm

5.3 Performance

Mention computer hardware specification – mention GPU not strictly relevant

Chapter 6

Deep Q-Learning

Chapter 7

Communicative Deep Q-Learning

Chapter 8

Supplementary Analysis

Chapter 9

Conclusions

9.1 Achievements

Summarise the achievements to confirm the project goals have been met.

9.2 Evaluation

Evaluation of the work (this may be in a separate chapter if there is substantial evaluation).

9.3 Future Work

How the project might be continued, but don't give the impression you ran out of time!

Bibliography

[Tha17] Thames Leisure. 12 amazing facts about London. *Thames Leisure*, archived January 7, 2017. Available at: https://web.archive.org/web/20170107005431/http://www.thamesleisure.co.uk/12-amazing-facts-london/.

[ASC17] Federal Highway Administration. EDC-1: Adaptive Signal Control Technology. *Federal Highway Administration*, 2017. Available at: https://www.fhwa.dot.gov/innovation/everydaycounts/edc-1/asct.cfm.

[HW08] M. Haklay and P. Weber. OpenStreetMap: User-generated street maps. *IEEE Pervasive Computing*, 7(4): 12–18, 2008. doi:10.1109/MPRV.2008.80.

[HWC] Department for Transport. The Highway Code: Using the road (Rules 159–203). *GOV.UK*. Available at: https://www.gov.uk/guidance/the-highway-code/using-the-road-159-to-203. Accessed: 20 February 2026.

[GS55] D. L. Gerlough and A. Schuhl. Use of Poisson Distribution in Highway Traffic: The Probability Theory Applied to Distribution of Vehicles on Two-Lane Highways. *Research Paper, Eno Foundation for Highway Traffic Control*, 1955. Available at: https://rosap.ntl.bts.gov/view/dot/16299. Accessed: 26 February 2026.

[GBG] S. Goel, S. F. Bus and C. Gershenson. Self-Organisation in Traffic Lights: Evolution of Signal Control with Advances in Sensors and Communications. *Research Paper, University at Albany*. Available at: https://arxiv.org/pdf/1708.07188. Accessed: 1 March 2026.

[DfT23] Department for Transport. Know your traffic signs: Pedestrian, cycle and equestrian crossings. *GOV.UK*, 5 December 2023. Available at: https://www.gov.uk/government/publications/know-your-traffic-signs/pedestrian-cycle-and-equestrian-crossings. Accessed: 1 March 2026.

[Li23] J. Li, L. Peng, K. Hou, Y. Tian, Y. Ma, S. Xu and T. Z. Qiu. Adaptive signal control and coordination for urban traffic control in a connected vehicle environment: A review. *Digital Transportation and Safety*, 2(2): 89–111, 2023. doi:10.48130/DTS-2023-0008. Available at: https://maxapress.com/article/doi/10.48130/DTS-2023-0008. Accessed: 1 March 2026.

[ITS12] Institute for Transport Studies, University of Leeds. Urban traffic control systems: Evidence on performance. *Institute for Transport Studies, University of Leeds*, archived October 19, 2012. Available at: https://web.archive.org/web/20121019152217/http://www.konsult.leeds.ac.uk/private/level2/instruments/

[CU21] B. Cesme and T. Urbanik II. Signal Coordination: Are We Really Coordinating For All Users? *Kittelson & Associates, Inc.*, October 15, 2021. Available at: https://www.kittelson.com/ideas/pros-and-cons-of-signal-coordination/.

[SIIG04] Federal Highway Administration. Signalized Intersections: Informational Guide — Chapter 9: Intersection-Wide Treatments. *Federal Highway Administration*, August, 2004. Available at: https://www.fhwa.dot.gov/publications/research/safety/04091/09.cfm.

[PS64] P. Stewart. "I know it when I see it". *Jacobellis v. Ohio*, 1964. Available at: https://en.wikipedia.org/wiki/I_know_it_when_I_see_it.

[Tr26] German Aerospace Center (DLR). TraCI Protocol Specification *SUMO Documentation*, February 13, 2026. Available at: https://sumo.dlr.de/docs/TraCI/Protocol.html.

[Sta19] Statista Research Department. Average car and van occupancy in England from 2002 to 2018. *Statista*, July, 2019. Available at: https://www.statista.com/statistics/314719/average-car-and-van-occupancy-in-england/.

Appendix A

Other appendices, e.g., code listing

Put your appendix sections here